

有沒有可能chunk一份pdf文件，以頁做為chunk的單位

答案是：絕對可以，而且在某些場景下（例如簡報 PPT、表單），這反而是最好的策略。

但在您的專案（使用 BGE-Rerank + Milvus）中，如果直接採用「一頁 = 一個 Chunk」，會面臨一個**致命的技術陷阱**。

讓我為您分析優缺點，並提供安全的實作程式碼。

1. 「以頁為單位」的優缺點分析

✓ 優點 (Pros)

1. **溯源最精準**：當 AI 回答「資料來源：第 5 頁」時，使用者打開 PDF 翻到第 5 頁，內容一定在那裡。體驗非常好。
2. **邏輯完整性**：對於 PowerPoint 轉成的 PDF 或圖文並茂的型錄，一頁通常就是一個完整的概念，切開反而破壞邏輯。
3. **Metadata 管理簡單**：不用計算字元索引，直接 `page_num` 搞定。

✗ 致命風險 (Cons) —— 請務必注意！

1. **Embedding 模型的 Token 限制 (最嚴重)**：

- 大多數開源 Embedding 模型（如 all-MiniLM-L6-v2 或 BGE-base）的輸入上限是 **512 tokens**。
 - 一張密密麻麻的 A4 合約或論文，文字量可能高達 **1000~2000 tokens**。
 - **後果**：如果您強制一頁切一塊，模型會**直接截斷 (Truncate)** 第 512 token 之後的所有內容。這意味著**每一頁的後半段內容**，對檢索系統來說是「隱形」的。
2. **語意中斷**：如果一句話剛好跨頁（第 5 頁最後一句連到第 6 頁第一句），以頁切分會打斷這句話的語意。
-

2. 判斷標準：您的 PDF 是哪種？

- 情況 A：簡報、投影片、大圖少字的型錄
 - ➡ 非常適合「以頁為單位」。
 - 情況 B：合約、論文、小說、技術手冊 (純文字密集)
 - ➡ 不能直接以頁為單位（會爆 Token）。
 - ➡ 建議策略：「頁內分塊」(Intra-page Chunking)。也就是以頁為邊界，但如果該頁字數太多，就在頁內再切細。
-

3. 實作程式碼 (Python)

既然您的專案要 Demo，我提供一個**「智慧型按頁切分」**的程式碼。

這個邏輯是：**盡量以一頁為單位**，但如果該頁字數太多，就自動在該頁內進行切分。這樣既保留了「第幾頁」的精準度，又避開了 Token 限制。

您需要安裝：

Bash

```
pip install pymupdf langchain-text-splitters
```

程式碼範例：

Python

```
import fitz # PyMuPDF
from langchain_text_splitters import RecursiveCharacterTextSplitter

def process_pdf_by_page(pdf_path, chunk_size=500, chunk_overlap=50):
    """
    策略：以「頁」為基礎單位。
    如果單頁內容超過 chunk_size，則在該頁內進行二次切分。
    """
    doc = fitz.open(pdf_path)
    file_name = pdf_path.split("/")[-1] # 取得檔名

    all_chunks = []

    # 初始化切分器（用於處理字數過多的頁面）
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        separators=["\n\n", "\n", "。", "!", "?", " ", "", ""],
    )

    print(f"開始處理文件：{file_name}，共 {len(doc)} 頁")
```

```
for page_num, page in enumerate(doc):
    # 1. 提取該頁文字
    page_text = page.get_text().strip()

    if not page_text:
        continue # 跳過空白頁

    # 2. 檢查字數是否安全 (這裡簡單用字元數估算, 中文 1 char approx 0.6~1 to
    # 假設 chunk_size 是 500, 我們設一個寬限值, 例如 600 字以內就不切
    if len(page_text) < chunk_size * 1.2:
        # --- 情況 A: 頁面內容適中, 直接整頁當作一個 Chunk ---
        chunk_metadata = {
            "file_name": file_name,
            "page_number": page_num + 1, # 人類習慣從 1 開始
            "chunk_id": f"{file_name}_p{page_num+1}_full"
        }
        all_chunks.append({
            "text": page_text,
            "metadata": chunk_metadata
        })
    else:
        # --- 情況 B: 頁面內容太多 (例如合約), 在「頁內」進行切分 ---
        # 注意: 我們只切分這一頁的內容, 不會跨到下一頁, 確保 metadata 的 page_
        sub_chunks = text_splitter.split_text(page_text)

        for i, sub_text in enumerate(sub_chunks):
            chunk_metadata = {
                "file_name": file_name,
                "page_number": page_num + 1,
                "chunk_id": f"{file_name}_p{page_num+1}_part{i+1}" #
            }
            all_chunks.append({
                "text": sub_text,
```

```

        "metadata": chunk_metadata
    })

    print(f"處理完成，共生成 {len(all_chunks)} 個向量片段。")
    return all_chunks

# --- 測試與整合建議 ---
# 在您的主程式中，把原本的 chunking 邏輯換成這個 function
# chunks = process_pdf_by_page("test_contract.pdf")
# 然後把 chunks 存入 Milvus

```

4. 這段程式碼對您的專案有什麼幫助？

1. **解決 BGE/Embedding 限制**：透過 `if len(page_text) < chunk_size` 的判斷，保證了不會因為一頁字太多而被模型截斷。
2. **完美的 Metadata**：無論有沒有切分，每一個 Chunk 都有準確的 `page_number`。
3. **提升 RAG 效果**：
 - 當 LLM 引用時，它可以說：「根據 **[合約.pdf 第3頁]** 的內容...」。
 - 即使第 3 頁被切成了 3 塊，它們的 Metadata 都是第 3 頁，對使用者來說是無感的。

建議：為了下週二的 Demo 穩定性，建議將 `chunk_size` 設定在 **400~500** 左右（配合常見 Embedding 模型限制），這樣最安全。

如果是chat with xls

處理 Excel (.xls/.xlsx) 檔案與 PDF 完全不同。

PDF 是「文章」，Excel 是「結構化數據」。如果你直接把 Excel 轉成文字然後切塊 (Chunking)，你會得到一堆沒有意義的數字和逗號，AI 會完全看不懂那是什麼。

為了趕在下週二 Demo，我們要用最穩健的策略：「**列 (Row) 序列化**」策略。

1. Excel 的挑戰與對策

- **挑戰**：向量資料庫看不懂表格的「上下左右」關係。如果切塊切斷了，AI 看到一個數字 5000，它不知道這是「薪水」還是「員工人數」。
 - **對策：Header Injection (欄位名注入)**。
 - 我們不能只存 [王小明, 30, 工程師]。
 - 我們要把它轉換成一段**有語意的文字**：姓名：王小明，年齡：30，職位：工程師。
 - **Chunk 單位**：通常 1 列 (Row) = 1 Chunk。
-

2. 實作程式碼 (Python + Pandas)

請安裝 Pandas：

Bash

```
pip install pandas openpyxl xlrd
```

將這段程式碼整合進你的資料處理流程中。這個函數會把 Excel 變成適合 Milvus 儲存的格式。

Python

```
import pandas as pd

def process_excel(file_path):
    """
    策略：將每一列 (Row) 轉換成一段「Key-Value」格式的文字。
    """
    # 1. 讀取 Excel
    # keep_default_na=False 避免將空字串轉成 NaN，方便處理
    try:
        df = pd.read_excel(file_path, keep_default_na=False)
    except Exception as e:
        print(f"讀取 Excel 失敗: {e}")
        return []

    file_name = file_path.split("/")[-1]
    all_chunks = []

    print(f"正在處理 Excel: {file_name}，共 {len(df)} 筆資料")

    # 2. 遍歷每一列 (Row)
    for index, row in df.iterrows():
        # --- 關鍵步驟：序列化 (Serialization) ---
        # 我們把這一系列的資料，組合成一段人類跟 AI 都讀得懂的文字
        # 例如： "產品名稱: iPhone 15, 價格: 30000, 庫存: 50"
```

```

row_text_parts = []
for col_name, cell_value in row.items():
    # 過濾掉空白的值，節省 token
    if cell_value == "" or pd.isna(cell_value):
        continue

    # 組合 "欄位名: 值"
    row_text_parts.append(f"{col_name}: {str(cell_value)}")

# 用逗號或換行連接
full_row_text = " | ".join(row_text_parts)

# 3. 加上一點上下文 (Optional)
# 告訴 AI 這是一筆什麼樣的資料
final_text = f"【資料來源: {file_name} 第 {index+2} 列】\n{full_row_

# 4. 準備 Chunk 結構
chunk_metadata = {
    "file_name": file_name,
    "page_number": 1, # Excel 通常沒有頁碼，統一設為 1 或用 Sheet Nam
    "row_number": index + 2, # Excel 通常第一列是標題，數據從第2列開始
    "chunk_id": f"{file_name}_row_{index}"
}

all_chunks.append({
    "text": final_text,
    "metadata": chunk_metadata
})

print(f"處理完成，共生成 {len(all_chunks)} 個向量片段。")
return all_chunks

```

```

# --- 測試用 ---

```

```

# if __name__ == "__main__":

```



```
# # 假設你有一個 'employees.xlsx'  
# chunks = process_excel("employees.xlsx")  
# print(chunks[0]['text'])  
# # 輸出會像： 【資料來源: employees.xlsx 第 2 列】  
# # 姓名: Jason | 部門: 研發部 | 分機: 1234
```

3. 這個方法能解決什麼問題？(Demo 亮點)

檢索準確度大增 (Better Recall)

當使用者問：「研發部 Jason 的分機是多少？」

- **Milvus 搜尋**：因為你的 Chunk 裡面明確寫著 姓名: Jason 和 部門: 研發部，向量相似度會非常高。
- **Rerank 過濾**：BGE-Rerank 會看到這段文字完全符合問題的邏輯，給出高分。
- **Ollama 回答**：因為文字格式很乾淨，Ollama 不會看錯行。

來源追溯 (Grounding)

- Chunk 裡記錄了 row_number。
 - AI 回答時可以標註：（來源: employees.xlsx, 第 5 列）。
 - 這對於會計、庫存管理等需要精確對照的場景非常有說服力。
-

4. ⚠ 重要限制與警告 (Demo 前必讀)

RAG 加上 Excel 有一個**天生弱點**，您必須在 Demo 時巧妙避開，或者設定好預期管理：

弱點：統計與計算 (Aggregation)

- 場景：如果使用者問「請問研發部的薪水總和是多少？」
- 為什麼會失敗：
 1. 檢索不全：Milvus 只會撈出 Top 20 筆最相關的列。如果研發部有 500 人，只有 20 人會被撈出來。
 2. LLM 算數爛：就算撈出 20 人，叫 LLM 心算加總 20 個數字，Llama 3 很有可能會算錯。

Demo 建議策略：

- 專攻「查找 (Lookup)」類問題：
 -  「請問 iPhone 15 的庫存剩多少？」
 -  「業務部的經理是誰？」
 -  「訂單編號 20240101 的客戶是哪家公司？」
- 避開「統計 (Analysis)」類問題：
 -  「全部產品的平均價格是多少？」
 -  「上個月總銷售額是多少？」

(如果真的要作統計，需要用 Text-to-SQL 或 Pandas Agent 技術，那會大幅增加架構複雜度，下週二來不及的，建議先守住「精準查找」這個強項即可。)

這是一個非常棒的問題。如果您在做 RAG，**BGE-M3** 是目前開源界公認的「全能型選手」，也被稱為「六邊形戰士」。

簡單來說，如果之前的模型是單一功能的「專科醫生」，BGE-M3 就是一個精通多國語言、能看長篇大論、還懂多種診斷技術的「超級醫生」。

BGE-M3 全名是 **BAAI General Embedding - M3**。這裡的 **M3** 代表它的三大核心能力：

1. **Multi-Linguality**（多語言能力）
2. **Multi-Functionality**（多功能性/混合檢索）
3. **Multi-Granularity**（多粒度/長文本支援）

以下我為您詳細拆解這三個 M，並說明為什麼它對您的專案很重要。

1. M1: Multi-Linguality (多語言能力)

- **痛點：** 以前的模型（如 OpenAI 的舊版 embedding）通常英文很好，中文普通，遇到小語種直接掛掉。
- **BGE-M3 能力：** 它在訓練時使用了 **100 多種語言** 的資料。
- **對您的意義：** 您的 PDF 如果是中文、英文混雜，甚至包含日文參考資料，BGE-M3 都能完美理解它們的語意關聯，不會出現「英文搜不到中文」的尷尬情況。

2. M2: Multi-Functionality (多功能性 - 最強大之處)

這是 BGE-M3 與傳統模型最大的不同。它不只產生一種向量，它可以同時執行三種檢索模式：

A. 稠密檢索 (Dense Retrieval) - 傳統強項

- **原理：** 把句子變成一個向量 (Embedding)。
- **功能：** 找「語意相似」的內容。
- **例子：** 搜「蘋果」，能找到「iPhone」或「水果」。
- **缺點：** 有時候會忽略精確關鍵字（例如型號）。

B. 稀疏檢索 (Sparse Retrieval / Lexical) - 補足弱點

- **原理：** 類似 BM25，計算關鍵字的權重。
- **功能：** 找「關鍵字完全匹配」的內容。
- **例子：** 搜「訂單編號 T-9527」，它會精準鎖定 "T-9527"，不會只給你找「訂單」。
- **對您的意義：** 這對您的 **Excel 專案** 特別重要！因為 Excel 裡的型號、人名需要精確匹配，不能只有模糊的語意。

C. 多向量檢索 (Multi-Vector / ColBERT) - 精細比對

- **原理：** 不把整句壓縮成一個向量，而是保留每個 Token 的向量進行細粒度互動。
- **功能：** 處理複雜邏輯，效果最準，但比較佔空間。

👉 **比喻：** BGE-M3 就像一把**瑞士軍刀**。以前你只有一把刀（Dense），現在這把刀還附帶了剪刀（Sparse）和螺絲起子（Multi-Vector），讓你一次滿足所有需求。

3. M3: Multi-Granularity (多粒度 / 長文本)

- **痛點：**傳統 BERT 模型（如 bert-base-chinese ）只能讀 **512 個 Token**（約 300-400 中文字）。超過就被切掉。
 - **BGE-M3 能力：**支援高達 **8192 個 Token**。
 - **對您的意義：**您的 PDF 如果有長篇合約或長表格，不需要切得碎碎的。BGE-M3 可以一口氣讀完很長的段落，保持上下文完整性。
-

實際應用：在您的專案中如何使用？

如果您想發揮 BGE-M3 的最大威力（混合檢索），推薦使用官方的 FlagEmbedding 套件，而不是只用 sentence-transformers （後者通常只用到 Dense 部分）。

安裝

Bash

```
pip install -U FlagEmbedding
```

Python 程式碼範例 (混合檢索 Hybrid Retrieval)

這段程式碼展示了它如何同時抓取「語意」和「關鍵字」：

Python

```
from FlagEmbedding import BGEM3FlagModel

# 1. 初始化模型
# use_fp16=True 可以省顯存並加速
print("正在載入 BGE-M3...")
model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)

# 2. 準備資料
sentences_1 = ["什麼是 BGE-M3 模型？"]
sentences_2 = [
    "BGE-M3 是一個支援多語言的強大模型。", # 語意相關
    "BM25 是一種基於關鍵字的演算法。",      # 不相關
    "M3 模型支援 8192 長度。"                # 關鍵字匹配
]

# 3. 產生 Embedding (同時產生 Dense, Sparse, ColBERT)
# 這是 BGE-M3 的獨門絕活
embeddings_1 = model.encode(sentences_1, return_dense=True, return_sparse=True)
embeddings_2 = model.encode(sentences_2, return_dense=True, return_sparse=True)

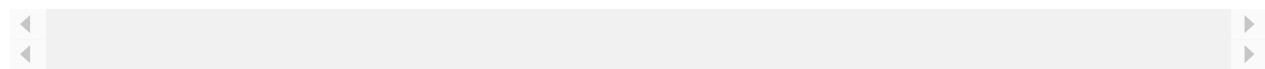
# 4. 計算分數 (混合分數)
# 它可以同時考量語意相似度和關鍵字匹配度
similarity = model.compute_score(embeddings_1, embeddings_2, weights_for_
# weights 對應 [Dense, Sparse, ColBERT] 的權重

print("混合檢索分數:", similarity)
```

BGE-M3 vs. BGE-Reranker-v2-m3 (不要搞混！)

這兩個名字很像，但角色不同：

模型名稱	角色	用途	你的專案位置
BAAI/bge-m3	Retriever (檢索者)	把文字轉成向量，存入 Milvus。	第一階段 (Milvus 搜尋用)
BAAI/bge-reranker-v2-m3	Reranker (重排序者)	把找出來的結果進行精準評分。	第二階段 (Python 腳本過濾用)



最佳拍檔建議： 在您的專案中，最頂級的配置是：

1. **Milvus 存入：**使用 BAAI/bge-m3 產生的向量（可以只用 Dense 向量以簡化開發）。
2. **Python 過濾：**使用 BAAI/bge-reranker-v2-m3 進行重排序。

這就是目前開源界最強的 **"BGE 全家桶"** 組合。

若使用 BAAI/bge-m3 則chunk要設為多少比較好

這是一個非常關鍵的參數設定。既然您已經決定升級使用 **BAAI/bge-m3**，您的 Chunk 設定策略可以比傳統模型（只能吃 512 tokens）**更大膽且靈活**。

直接給您結論：對於大多數的 RAG 應用，搭配 BGE-M3 的**黃金甜蜜點是 1000 ~ 1200 tokens**（約 600~800 中文字）。

但因為您下週二就要 Demo，為了求穩並發揮 BGE-M3 的長文本優勢，我有以下三個具體的設定建議：

1. 為什麼不用 512 了？(BGE-M3 的優勢)

- **傳統模型 (如 bge-base, OpenAI-ada-002)：**
 - 限制：通常是 512 tokens。
 - 問題：一張 A4 紙如果是滿的，通常會超過 512。如果你設 512，這頁就會被切成兩半，語意斷裂。
- **BGE-M3：**
 - 能力：最大支援 **8192 tokens**。
 - 優勢：它可以一口氣讀完**整整 5~10 頁**的 PDF 都不會被截斷。

2. 推薦設定：針對您的 PDF 專案

方案 A：安全平衡型 (推薦 Demo 使用) 🌟

這是最穩健的設定，既能保持段落完整，又不會讓向量過度稀釋。

- **Chunk Size: 1024 (Tokens)**
 - 約等於 600~800 個中文字。
 - 涵蓋範圍：大約是 **1 到 1.5 頁**標準 A4 文件的內容。

- **Chunk Overlap: 200**
 - *重疊部分：確保切分點的語意有被保留。*

為什麼選 1024？

1. **完整性**：剛好能包住大部分的完整章節或條款，不會把「第一條...」跟「...的罰則」切開。
2. **LLM 友善**：如果您檢索 Top 3 個 chunks 給 Llama 3，總共約 3000 tokens，這在 Llama 3 (8k context) 的舒適圈內，回答速度快。

方案 B：整頁切分型 (搭配您剛才的想法)

既然 BGE-M3 支援長文本，您剛才提到的「以頁為單位」現在變得**非常可行**。

- **Chunk Size: 2000** (Tokens)
 - *這個大小足以容納即使是字體極小的密集排版 A4 頁面。*
- **Chunk Overlap: 0** (或極小)
 - *因為是以頁為單位，通常頁與頁之間有自然的分隔。*

適用場景：您的 PDF 是簡報、有明確頁碼對應的合約。 **風險**：如果某一頁講了三個完全不同的主題，轉成一個向量後，特徵會比較模糊。

3. Python 程式碼調整 (LangChain)

在您的資料處理腳本中，請這樣調整 `RecursiveCharacterTextSplitter`：

Python

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 針對 BGE-M3 的最佳化設定
text_splitter = RecursiveCharacterTextSplitter(
    # 1. 設定為 1024 (比傳統 512 大一倍，發揮 M3 優勢)
    chunk_size=1024,

    # 2. 設定 15%~20% 的重疊，避免切斷關鍵句
    chunk_overlap=200,

    # 3. 設定長度計算函數 (重要!)
    # 預設是算「字元數」(len)，但在中文裡 1 char != 1 token
    # 為了保險，我們通常假設 1 token ≈ 1.5 chars (英文) 或 0.7 chars (中文)
    # 簡單起見，直接用 len 也可以，但在這裡把 chunk_size 設大一點容錯
    length_function=len,

    # 4. 分隔符優先順序
    separators=["\n\n", "\n", "。", "!", "?", ";", " ", ""]
)

# ... 接續讀取 PDF 的邏輯 ...
```

4. 補充：為什麼不設 8192 (最大值)？

您可能會問：「既然它支援 8192，我為什麼不直接把半本書切成一塊？」

這涉及一個向量檢索的物理限制：「**向量稀釋 (Vector Dilution)**」。

- 比喻：

- **Chunk = 500 (短)**：像是一杯濃縮咖啡，味道（特徵）很強烈。搜「咖啡」，馬上找到。
- **Chunk = 8000 (長)**：像是一桶水裡滴了一滴咖啡。整桶水（向量）的味道很淡。搜「咖啡」，可能找不到這桶水，因為它還有茶味、果汁味（其他主題）混在一起。

結論： 為了下週二的 Demo 效果，請使用 **Chunk Size = 1024**。這個大小能讓 **BGE-M3** 發揮「讀得懂上下文」的優勢，同時保持 **Milvus** 搜尋時的精準度。
若改用BGE-M3，milvus collection要進行調整

是的，一定要調整，而且這是毀滅性的調整（必須刪除舊的 Collection 重建）。

因為 Milvus 的 Collection 一旦建立，**向量維度 (Dimension)** 就固定了，無法修改。

大多數常見模型（如 all-MiniLM-L6-v2）是 **384** 維，OpenAI 是 **1536** 維，而 **BAAI/bge-m3** 的稠密向量 (Dense Vector) 是 **1024** 維。

為了讓您下週二的 Demo 順利，請依照以下步驟修改您的 `database_layer.py` 或初始化程式碼：

1. 關鍵修改點：維度 (dim) 變更為 1024

BGE-M3 產出的 Dense Embedding 預設長度是 **1024**。如果您把 1024 維的資料硬塞進原本 384 維的 Collection，Milvus 會直接報錯。

2. 實際 Python 程式碼 (重建 Schema)

這是您在 `database_layer.py` 初始化 Milvus 時應該要執行的程式碼。請注意我標註 ★ 的地方：

Python

```
from pymilvus import connections, FieldSchema, CollectionSchema, DataType

# 連線設定 (維持不變)
connections.connect("default", host="192.168.1.X", port="19530")

COLLECTION_NAME = "pdf_chunks_collection"

def init_milvus_collection():
    # 1. 檢查是否已存在，若存在且維度不對，必須刪除！
    # 注意：這會清空所有資料，請確保您的 PDF 解析腳本可以重新跑一次
    if utility.has_collection(COLLECTION_NAME):
        print(f"偵測到舊的 Collection: {COLLECTION_NAME}，正在刪除以更新 Schema")
        utility.drop_collection(COLLECTION_NAME)

    # 2. 定義欄位
    fields = [
        # 主鍵 (Auto ID)
        FieldSchema(name="id", dtype=DataType.INT64, is_primary=True, auto_id=True),

        # 為了 BGE-M3，這裡的維度必須是 1024
        # *** 關鍵修改 ***
        FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=1024),

        # 儲存 Metadata (建議直接開啟動態 Schema，或是把文字存這裡)
        # 為了 Demo 方便，我們把文字長度設大一點，或者用 VARCHAR
        FieldSchema(name="chunk_text", dtype=DataType.VARCHAR, max_length=8192),
        FieldSchema(name="file_name", dtype=DataType.VARCHAR, max_length=256)]

    collection_schema = CollectionSchema(fields)
    collection = create_collection(COLLECTION_NAME, collection_schema)
```

```

        FieldSchema(name="page_number", dtype=DataType.INT64),
    ]

    schema = CollectionSchema(fields, description="Collection for BGE-M3")

    # 3. 建立 Collection
    collection = Collection(name=COLLECTION_NAME, schema=schema)

    # 4. 建立索引 (Index) - 為了搜尋速度
    # BGE-M3 建議使用 COSINE 或 IP (內積) 作為距離計算方式，效果比 L2 好
    index_params = {
        "metric_type": "COSINE", # ★ 建議改成 COSINE
        "index_type": "HNSW",    # 速度與準確度平衡最好的索引
        "params": {"M": 16, "efConstruction": 200}
    }
    collection.create_index(field_name="embedding", index_params=index_params)

    print(f"Milvus Collection {COLLECTION_NAME} 建立完成 (dim=1024) !")
    return collection

# 執行初始化
# init_milvus_collection()

```

3. 關於 BGE-M3 的「稀疏向量 (Sparse Vector)」

您可能讀過 BGE-M3 支援「稀疏檢索」(Hybrid Search)。Milvus 雖然支援稀疏向量 (`DataType.SPARSE_FLOAT_VECTOR`)，但我強烈建議您在下週二的 **Demo** 前不要開啟這個功能。

為什麼？

1. **複雜度暴增**：您需要同時儲存兩個向量欄位 (Dense + Sparse)。
2. **程式碼大改**：搜尋時需要寫特殊的 Hybrid Search 邏輯和權重分配 (rrf 等)。
3. **效益遞減**：單純使用 BGE-M3 的 **Dense (1024維)** 加上後端的 **Reranker**，已經足夠應付 99% 的 Demo 場景了。

策略建議：現在先用 BGE-M3 的 **Dense Vector (1024)** 模式就好，這已經比之前的模型強很多了。等 Demo 結束後，再來研究如何加入 Sparse Vector 做混合檢索。

總結操作步驟

1. 停止您的 Python 程式。
2. 確認程式碼中的 dim 已改為 **1024**。
3. 確認 metric_type 已改為 **COSINE** (推薦) 或 IP。
4. 執行程式，讓它觸發 utility.drop_collection (刪除舊庫)。
5. 重新執行您的 PDF 解析與向量化腳本 (Ingestion Script)，把資料重新灌進去。

我目前碰到最大的問題是，多pdf文件，要進行綜合摘要

這確實是 RAG 系統中最難的一關。

「**多文檔綜合摘要 (Multi-Document Summarization)**」之所以難，是因為標準的 RAG (檢索 -> 排序 -> 生成) 是為了「找針 (Fact Retrieval)」設計的，而不是為了「看草堆 (Global Understanding)」設計的。

如果您直接把 3 份 PDF 的內容全部丟給 LLM，會發生：

1. **爆 Token**：超過 Llama 3 的 Context Window。
2. **顧此失彼**：LLM 讀了後面忘記前面 (Lost in the Middle 現象)。

為了趕在下週二 Demo，我不建議上複雜的 Agent (如 LangGraph)，而是用**「分進合擊 (Map-Reduce)」加上「元數據過濾 (Metadata Filtering)」**的策略來解決。

以下是針對您的架構設計的解決方案：

核心策略：主題式切片比較 (Topic-Slicing)

不要試圖一次摘要「整份」文件。要把問題拆解成**「針對特定主題，橫向撈取各文件的片段，再進行融合」**。

情境：使用者問「請比較這三份保險文件的**理賠上限**和**自負額**。」

步驟 1：修改 Milvus 搜尋邏輯 (從「海選」變成「分組預選」)

傳統做法是全庫搜尋 Top 10，這會導致可能找到 8 個 A 文件的片段，B 和 C 卻沒被選到。**新做法**：針對每一份 PDF，**分別**去撈 Top 3 相關片段，強迫每一份文件都有「發言權」。

步驟 2：實作 Python 程式碼

請在您的 `database_layer.py` 或 `rag_engine.py` 中加入這個專門處理多文件摘要的函數。

Milvus 的關鍵語法是 `expr` (過濾表達式)。

Python

在 `rag_engine.py` 中新增

```
def multi_file_summarize(self, query, target_files):
    """
    target_files: list of strings, e.g. ["A.pdf", "B.pdf", "C.pdf"]
    query: 使用者的問題，例如 "比較理賠上限"
    """
    print(f"正在進行多文件綜合摘要，目標文件數: {len(target_files)}")

    aggregated_chunks = []

    # --- 階段一：分進 (Map) ---
    # 強制讓每一份文件都有機會被檢索到
    for file_name in target_files:
        print(f"-> 正在檢索文件: {file_name} ...")

        # 1. Milvus 搜尋 (帶有 filter)
        # 使用 expr 鎖定特定檔名
        search_params = {"metric_type": "COSINE", "params": {"nprobe": 10}}

        # 注意：這裡假設 Milvus 裡有存 'file_name' 這個 scalar field
        filter_expr = f'file_name == "{file_name}"'

        query_vector = self.embed_model.encode([query])[0].tolist()
```



```

results = self.db.milvus_col.search(
    data=[query_vector],
    anns_field="embedding",
    param=search_params,
    limit=3, # 每份文件只取最精華的 3 個片段
    expr=filter_expr, # ★關鍵：只搜這份文件
    output_fields=["chunk_text", "page_number"]
)

# 2. 收集結果
for hits in results:
    for hit in hits:
        aggregated_chunks.append({
            "text": hit.entity.get("chunk_text"),
            "file_name": file_name,
            "page": hit.entity.get("page_number"),
            "score": hit.score
        })

# --- 階段二：過濾與重排序 (Rerank) ---
# 現在 aggregated_chunks 裡面可能有 3份文件 * 3片段 = 9 個片段
# 我們把它們全部丟進 BGE-Rerank 再洗牌一次，確保最準的排前面
print(f" -> 彙整共 {len(aggregated_chunks)} 個片段，進行 Rerank...")

pairs = [[query, doc['text']] for doc in aggregated_chunks]
scores = self.reranker.predict(pairs)

# 更新分數並排序
for i, score in enumerate(scores):
    aggregated_chunks[i]['rerank_score'] = score

# 取出前 Top K (例如總共取 5~6 個片段，避免 Context 太長)
final_docs = sorted(aggregated_chunks, key=lambda x: x['rerank_score'])

```

```
# --- 階段三：合擊 (Reduce / Synthesis) ---
# 組裝特殊的 Prompt
context_str = ""
for doc in final_docs:
    context_str += f"【文件：{doc['file_name']} (第{doc['page']}頁)】\n
```

```
final_prompt = f"""
```

你是一個專業的分析師。請根據以下來自多份文件的資料，針對問題進行「綜合比較與摘要」。

【重要規則】：

1. 請不要分別摘要每一份文件，請嘗試將它們的資訊「融合」在一起。
2. 如果可以，請使用「表格」或「條列式」呈現不同文件之間的差異。
3. 清楚標示資訊來源。

【參考資料集】：

{context_str}

【使用者問題】：

{query}

"""

```
# 呼叫 Ollama
response = self.ollama_client.chat(model=OLLAMA_MODEL, messages=[
    {'role': 'user', 'content': final_prompt},
])

return response['message']['content']
```

3. Demo 時的技巧 (Cheat Sheet)

要在 Demo 展現「多文件摘要」的威力，使用者的 **Prompt (提問方式)** 也很重要。您可以引導評審或自己 Demo 時這樣操作：

技巧 A：指定文件範圍

不要讓程式自己猜要搜哪些檔案（那樣很難寫）。在 UI 上（或指令列）設計一個步驟，讓使用者先**勾選**：「我要針對這三份 PDF 進行提問」。

- 程式碼收到 `target_files = ["A.pdf", "B.pdf"]` 。
- 然後呼叫上面的 `multi_file_summarize` 。

技巧 B：針對性提問 (Comparison)

-  **壞問題**：「請摘要這些文件。」(太廣泛，RAG 很難抓到重點，除非您抓每份文件的第一頁 Intro)。
-  **好問題**：「請比較這三份合約的**付款條件**和**違約金**。」
 - 這會觸發 Milvus 精準抓出「付款」和「違約」相關的段落。
 - LLM 就能完美寫出：「A 合約是 30 天結算，B 合約是 60 天...」。

技巧 C：如果不幸被問到「請摘要整份文件」怎麼辦？

如果 Demo 必須回答「這份文件在講什麼？」這種廣泛問題，RAG 檢索通常會失效（因為它只會抓到某個細節）。

急救解法 (Hard-coded Rule)：在程式邏輯裡偷加一條規則：如果 `query` 包含「摘要」、「大綱」、「總結」等關鍵字：

- **不要搜 Milvus**。

- 直接去 MySQL/Disk 讀取該 PDF 的 **第一頁 (Page 1)** 和 **最後一頁** (通常是摘要和結論)。
- 把這兩頁餵給 LLM 說：「請根據這兩頁寫出摘要。」

這招雖然暴力，但在 Demo 時效果非常好，且速度極快。

總結下週二的對策

1. **資料面**：確保 Milvus schema 裡有 `file_name` 欄位，這是做多文件過濾的基礎。
2. **演算法面**：採用 "**For-Loop Search**" (對每份文件單獨搜 Top 3)，而不是全庫搜 Top 10。
3. **生成面**：Prompt 強調「融合」與「比較」，最好要求輸出 Markdown 表格。

這套「指定文件 + 分組檢索」的邏輯，是目前最符合您現有架構且能在短期內實現的方案。